MODULE 08 · BLOCK 2 - CONTROL FLOW - CONDITIONAL BLOCKS AND LOOPS

for loops and range()

Every form of range(), counting passes, looping over strings, lists and dictionaries, and the traps in changing what you loop over.

BLOCK 2 of 4 EXAM WEIGHT 29% MODULE 08 of 28

admin@sthithapragna.com

A for loop runs once for each item in a sequence. range() builds a sequence of whole numbers that **includes start and stops just before stop**.

admin@sthithapragna.com

for name in sequence:

```
for i in range(3):
    print(i)            # 0, then 1, then 2
print("done")
```

1. Each pass, the next item from the sequence is assigned to i. Then the body runs.
2. No update line and no test to forget: the sequence decides how many passes.

admin@sthithapragna.com

range(stop), range(start, stop), range(start, stop, step)

CALL	VALUES	RULE
`range(5)`	0, 1, 2, 3, 4	One argument is the stop. Start is 0.
`range(2, 5)`	2, 3, 4	Two arguments are start and stop.
`range(0, 10, 3)`	0, 3, 6, 9	The third argument is the step.
`range(5, 0, -1)`	5, 4, 3, 2, 1	A negative step counts down. Stop is still excluded.

THE RULE

The stop value is never included, whichever direction you count.

admin@sthithapragna.com

list() shows the values. print() alone does not

```
1  print(list(range(2, 5)))       # [2, 3, 4]
2  print(list(range(0, 10, 3)))   # [0, 3, 6, 9]
3  print(list(range(-1, 1)))      # [-1, 0]
4  print(list(range(10, 0, -3)))  # [10, 7, 4, 1]
5  print(range(3))                # range(0, 3)
```

3 Negative starts are fine. From -1 up to, but not including, 1.

5 A range is not a list. Printing it shows the range itself, not its values.

admin@sthithapragna.com

When the step points away from stop, nothing happens

```
1  print(list(range(5, 0)))       # []  step is +1, cannot go down
2  print(list(range(0, -2)))      # []
3  print(list(range(2, 2)))       # []  start equals stop
4  print(list(range(0, 5, -1)))   # []  step is -1, cannot go up
```

1-2 Without a step, range only counts up. A lower stop gives an empty range.

3-4 An empty range is not an error. The loop body simply runs zero times.

admin@sthithapragna.com

How many times does the body run?

RANGE	VALUES	PASSES
`range(10)`	0 to 9	10
`range(1, 10)`	1 to 9	9: stop minus start
`range(0, 10, 2)`	0, 2, 4, 6, 8	5
`range(1, 10, 3)`	1, 4, 7	3
`range(10, 0, -3)`	10, 7, 4, 1	4

EXAM TECHNIQUE

With a step of 1, passes are stop minus start. With any other step, write the values out.

The calls that fail

CALL	ERROR
`range(1.5)`	TypeError: 'float' object cannot be interpreted as an integer
`range("5")`	TypeError: 'str' object cannot be interpreted as an integer
`range(0, 10, 0)`	ValueError: range() arg 3 must not be zero
`for i in 5:`	TypeError: 'int' object is not iterable

ON THE EXAM

range() takes whole numbers only. A step of zero is the one ValueError.

admin@sthithapragna.com

A for loop can walk any sequence

```
1  for ch in "Hi!":
2      print(ch, end=" ")        # H i !
3  scores = {"Ann": 7, "Bob": 9}
4  for name in scores:
5      print(name)               # Ann, then Bob: the keys
6  for name, s in scores.items():
7      print(name, s)            # Ann 7, then Bob 9
```

1-2 A string is a sequence of characters, one per pass.

4-5 Looping over a dictionary gives its keys, in insertion order.

6-7 items() gives key and value pairs, unpacked into two names.

admin@sthithapragna.com

for keeps the last value. while goes one further

FOR

Last value used

```
for i in range(3):
    pass
print(i)
```

Prints 2, the last value assigned. range(3) never produced 3.

WHILE

The value that failed the test

```
i = 0
while i < 3:
    i += 1
print(i)
```

Prints 3: the loop only stops once i has reached 3.

ON THE EXAM

If a for loop runs zero times, its variable is never assigned: print(i) would be a NameError.

admin@sthithapragna.com

Changing i inside the body does not steer the loop

```
1  for i in range(3):
2      print(i, end=" ")
3      i = 10                    # overwritten on the next pass
4  print()                       # output so far: 0 1 2
```

3 The next pass assigns the next value from range, whatever i was set to.

admin@sthithapragna.com

The loop variable is a copy of the name, not a window into the list

LOOP OVER THE ITEMS

The list does not change

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n *= 2
4  print(nums)
```

[1, 2, 3]. n is rebound to a new number; the list is untouched.

LOOP OVER THE POSITIONS

The list changes

```
1  nums = [1, 2, 3]
2  for i in range(len(nums)):
3      nums[i] *= 2
4  print(nums)
```

[2, 4, 6]. nums[i] names a slot in the list itself.

ON THE EXAM

To change a list's elements, loop over range(len(the_list)) and assign through the index.

admin@sthithapragna.com

Removing items as you go skips some

```
1  data = [1, 2, 3, 4]
2  for x in data:
3      data.remove(x)
4  print(data)                  # [2, 4]
5  li = [1, 2, 3]
6  for v in range(len(li)):     # range(3), fixed now
7      li.insert(1, li[v])
8  print(li)                    # [1, 1, 1, 1, 2, 3]
```

2-3 The loop tracks a position. Removing an item shifts the rest left, so the next one is jumped over.

6 range(len(li)) is worked out once, before the first pass. Growing li does not add passes.

admin@sthithapragna.com

else runs when the loop was not broken

```
1  for n in [3, 5, 7]:
2      if n % 2 == 0:
3          print("found even")
4          break
5  else:
6      print("no even numbers")   # this runs
```

4 A break here would skip the else.

5-6 Exactly as with while: no break, so the else runs. Even after zero passes.

Building a result pass by pass

```
1  total = 0
2  for i in range(1, 5):
3      total += i                  # 1 + 2 + 3 + 4
4  print(total)                    # 10
5  word = ""
6  for ch in "abc":
7      word = ch + word            # new character goes in front
8  print(word)                     # cba
```

1-3 Start with an empty result, and add to it on every pass.

7 Order matters: ch + word builds backwards, word + ch builds forwards.

for versus while

FOR

Walk a known sequence

- The items or count are known up front
- No condition and no update to forget
- Cannot run forever over a range
- Loop variable keeps its last value

WHILE

Repeat until something changes

- The number of passes is not known
- You write the test and the update
- Can run forever if you get it wrong
- Ideal for reading input to a sentinel

ON THE EXAM

Both support break, continue and else, with exactly the same rules.

What to carry into the exam

- range includes start and **excludes stop**, in both directions.
- No step means +1. range(5, 0) is empty, not an error.
- After a for loop, the variable holds its **last** value: range(3) leaves 2.
- Reassigning the loop variable does not change the list or the passes.
- range(len(x)) is fixed at the start. Removing items while looping skips some.

How many stars does this program print?

```
1  for i in range(-2, 3, 2):
2      print("*")
```

A Two

B Four

C Three

D Five

◆ PAUSE AND DECIDE

admin@sthithapragna.com

How many stars does this program print?

```
1  for i in range(-2, 3, 2):
2      print("*")
```

Values: -2, 0, 2. The next would be 4, which is past the stop.

A Two

Stops at 0. 2 is still below the stop of 3.

B Four

Includes 4. It is beyond the stop.

C Three

-2, 0 and 2 are all below 3.

D Five

Counts every integer from -2 to 2. The step is 2, not 1.

THE SIGNAL

With a step other than 1, list the values. Stop when the next would reach or pass stop.

What is the output of this program?

```
1  for i in range(10, 0, -3):
2      print(i, end=" ")
```

A 10 7 4

B 10 7 4 1

C 9 6 3 0

D 7 4 1

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the output of this program?

```
1  for i in range(10, 0, -3):
2      print(i, end=" ")
```

10, 7, 4, 1. The next would be -2, which is past the stop of 0.

A `10 7 4`

Stops early. 1 is still above the stop of 0.

B `10 7 4 1`

Start is included; keep stepping while above 0.

C `9 6 3 0`

Starts at 9 and includes 0. Start is included; stop is not.

D `7 4 1`

Skips the start. The first value is always start itself.

THE SIGNAL

Start is always the first value. Stop is never reached, in either direction.

admin@sthithapragna.com

What is the output of this program?

```
1  total = 0
2  for k in range(1, 6, 2):
3      total += k
4  print(k, total)
```

A 6 9

B 7 9

C 5 9

D 5 15

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the output of this program?

```
1  total = 0
2  for k in range(1, 6, 2):
3      total += k
4  print(k, total)
```

k takes 1, 3, 5. total is 1 + 3 + 5 = 9. k keeps its last value, 5.

A 6 9

Thinks k ends at the stop. range never produces 6.

B 7 9

Adds one more step, as a while counter would.

C 5 9

k keeps the last value it was given; 1 + 3 + 5 is 9.

D 5 15

Adds every number from 1 to 5. The step is 2.

THE SIGNAL

After a for loop, the variable holds the last value the sequence produced, never the stop.

admin@sthithapragna.com

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n = n * 10
4  print(nums)
```

A `[10, 20, 30]`

B `[1, 2, 3]`

C `[10, 2, 3]`

D The code raises a TypeError

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  nums = [1, 2, 3]
2  for n in nums:
3      n = n * 10
4  print(nums)
```

n is moved to a new number each pass. The list's slots are never touched.

A `[10, 20, 30]`
Would need nums[i] = … inside a range(len(nums)) loop.

B `[1, 2, 3]`
Assigning to n rebinds the name only. The list is unchanged.

C `[10, 2, 3]`
No pass changes the list, including the first.

D The code raises a TypeError
Multiplying an int by 10 is fine.

THE SIGNAL

Assigning to the loop variable never changes the list. Assign through an index to do that.

What is the output of this program?

```
for i in range(5, 1):
    print("A")
else:
    print("B")
```

A. A four times, then B

B. Nothing is printed

C. A, then B

D. B

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  for i in range(5, 1):
2      print("A")
3  else:
4      print("B")
```

range(5, 1) counts up by 1 from 5, and 5 is already past 1: it is empty.

A. A four times, then B
Counts down without being told to. No step means +1.

B. Nothing is printed
The body never runs, but no break fired, so the else does.

C. A, then B
An empty range gives zero passes, not one.

D. B
Empty range: zero passes. No break: the else runs.

THE SIGNAL

Two rules together: no step means counting up, and a loop's else runs whenever no break fired.

admin@sthithapragna.com

What is the output of this program?

```
1  word = ""
2  for ch in "code":
3      word = ch + word
4  print(word)
```

A code

B cccc

C e

D edoc

◆ PAUSE AND DECIDE

admin@sthithapragna.com

What is the output of this program?

```
1  word = ""
2  for ch in "code":
3      word = ch + word
4  print(word)
```

"c", then "oc", then "doc", then "edoc".

A `code`

That is word + ch. Here each new character goes in front.

B `cccc`

ch takes a different character on each pass.

C `e`

word is built up, not replaced. It keeps the earlier characters.

D **`edoc`**

Each character is placed in front of what came before.

THE SIGNAL

ch + word builds in reverse; word + ch builds in order. Trace the first two passes to be sure.

What is the output of this program?

```
1  prices = {"tea": 2, "cake": 3}
2  total = 0
3  for item in prices:
4      total += prices[item]
5  print(total, item)
```

A 5 tea

B 5 cake

C The code raises a TypeError

D 2 tea

◆ PAUSE AND DECIDE

What is the output of this program?

```
1  prices = {"tea": 2, "cake": 3}
2  total = 0
3  for item in prices:
4      total += prices[item]
5  print(total, item)
```

item takes the keys "tea", then "cake". prices[item] looks up each value.

A `5 tea`

item keeps the last key, not the first.

B `5 cake`

Keys in insertion order; the last one is "cake". 2 + 3 is 5.

C The code raises a TypeError

item is a key, so prices[item] is a valid lookup.

D `2 tea`

The loop makes two passes, not one.

THE SIGNAL

Looping over a dictionary gives its keys, in insertion order. Look values up with d[key].

> Each range is converted to a list on its own.

Which two ranges produce exactly three values? (Choose two.)

A `range(3)`

B `range(1, 3)`

C `range(0, 9, 3)`

D `range(3, 0)`

◆ PAUSE AND DECIDE

admin@sthithapragna.com

Which two ranges produce exactly three values? (Choose two.)

A `range(3)`

0, 1, 2.

B `range(1, 3)`

1, 2: stop minus start is 2.

C `range(0, 9, 3)`

0, 3, 6. The next would be 9, the excluded stop.

D `range(3, 0)`

No step, so it counts up from 3: empty.

THE SIGNAL

For every range question: write the values out, then count them.

admin@sthithapragna.com